ECSE 324 Computer Organization

Laboratory 2

I/O

Seyed Yazdan Zinati

March 20, 2021

Abstract

In this laboratory, we implement drivers for I/O devices of the DE1-SoC computer in ARM assembly. These drivers include drivers for slider switches, LEDs, HEX displays, pushbuttons, and timers. We then use the mentioned drivers to write different programs; namely, we program stopwatches using polling and interrupts. Through these programs, we exercise our understanding of memory-mapped I/O, polling, and interrupts.

1 Basic I/O

In this section, we implement drivers for slider switches, LEDs, HEX displays, and pushbuttons.

Slider switches and LEDs

We use memory-mapped I/O to implement drivers to read from the slider switches and write to LEDs. The `read_slider_switches_ASM` subroutine reads from the slider switch data register and stores it into `R0`. On the other hand, the `write_LEDs_ASM` subroutine stores the value in `R0` into the LED data register. Both subroutines are similar as they save used registers onto the stack, load the data register addresses, read or write to them using word accesses, and branch back to the callee. As both use one-hot encoding, calling them sequentially will turn on LEDs corresponding to the state of the switches. We name this subroutine `switch_to_LED`.

HEX displays

We implement three drivers for the HEX displays. The three drivers receive display indices to write into through `R0`. `HEX_clear_ASM` turns off while `HEX_flood_ASM` turns all the segments of of the HEX displays. `HEX_write_ASM` receives an additional argument through `R1` indicating the HEX digit to display.

All three subroutines follow the same order. First, the display code to be written to the HEX display is determined. To do so, we load the display codes corresponding to hexadecimal digits with an immediate offset as they are stores as a byte array in consecutive memory locations. For instance, `LDRB R2, [R3, #5]` where `R3` is the byte array base address loads the display code corresponding to 5 into `R2`.

Then, we enter a loop that checks which displays were passed as arguments and writes to them. We keep track of the display index with a loop counter that starts from zero, and we perform a `TST` logical test to determine if the encoding bit corresponding to a display is on. At the end of our loop, we increment the loop counter and perform a logical shift left on the register used to perform the `TST` operation to check for the next display. Each time we encounter a display passed as an argument in our loop, we write to it by finding the corresponding data register base address and offset.

The main challenge in writing the HEX display drivers was understanding one-hot encoding and ensuring code conciseness and readability. We achieved a concise code using a byte array to find display codes instead of checking for all 16 values, a loop to find the displays to write to, and automatically finding the base address and offset for each display.

Push buttons

In this part, we write seven subroutines for pushbuttons. However, unlike LEDs, HEX displays, and slider switches, pushbuttons have interrupt mask and edge capture registers in addition to a data register.

Pushbutton read and write subroutines such as `read_PB_data_ASM`, `read_PB_edgecp_ASM`, and `PB_clear_edgecp_ASM` operate in a similar way to the previously described LED write and slider switch read subroutines. The two `PB_edgecp_is_pressed_ASM` and `PB_data_is_pressed_ASM` subroutines perform an `AND` operation between the subroutine argument and the return value of `read_PB_edgecp_ASM` and `read_PB_data_ASM` respectively. To enable and disable interrupts for the corresponding pushbuttons in `enable_PB_INT_ASM` and `disable_PB_INT_ASM`, we update the interrupt mask register with the result of `ORR` and `BitClear` operations between the subroutine input and the interrupt mask register respectively.

The main challenge faced in this part was manipulating bits using decscribed logical operations.

Simple I/O application

To test the drivers we wrote so far, we write an application that maps slider switched directly to LEDs, floods `HEX4` and `HEX5` displays, and displays the value set using slider switches `SW0-SW3` on the HEX displays corresponding to the pressed pushbuttons. Additionally, `SW9` clears all HEX displays.

This program executes in an infinite loop. We first call the previously described `switch_to_LED` subroutine. Then, we flood `HEX4` and `HEX5` displays by calling `HEX_flood_ASM` with the argument being the one-hot encoding for `HEX4` and `HEX5` displays. Next, we isolate `SW0-SW3` and `SW9` switches using an `AND` operation. We check if the `SW9` bit is on and call `HEX_clear_ASM` with `HEX0` to `HEX3` displays as arguments if the condition is true. Finally, we call the `HEX_write_ASM` subroutine by passing the pushbuttons and `SW0-SW3` switches as the display indices (`R0`) and hexadecimal digit to display (`R1`) respectively.

2 Timers

In this section, we write drivers for the ARM A9 private timer and use it to implement an up counter and a stopwatch that use polling.

We interact with three of the four registers of the ARM A9 private timer: the load, control, and interrupt status registers. The `ARM_TIM_config_ASM` subroutine receives two arguments. The first argument is the initial count that we write into the load register of the timer. The second argument is the control bits (`IAE` bits) and prescaler value which we write into the control registers. `ARM_TIM_read_INT_ASM` and `ARM_TIM_clear_INT_ASM simply` read and clear the interrupt status register similar to previously described drivers.

Up counter

We implement an up counter that counts in increments of one second from 0 to 15 and displays it on the `HEX0` display. The timer restarts when the value 15 is reached. The application flowchart is illustrated in Figure 1.

The ARM A9 private timer uses a clock frequency of 200Mhz. Therefore, to count in increments of 1 second, we use an initial count of 200'000'000. Moreover, we want to start the timer and automatically load the initial count when the timer reaches zero. To achieve this, we set both `A` and `E` bits in the control register to 1. We call the previously implemented `ARM_TIM_config_ASM` to configure the timer.

Next, we initialize a counter variable to zero and enter the polling loop. Inside the loop, we display the counter value into the `HEX0` display by calling `HEX_write_ASM` and poll the `F` bit of the interrupt status register to determine if the timer has reached zero. If the timer has not yet reached zero, we continue the loop. Otherwise, the counter gets incremented or set back to zero if the maximum count (15) has been reached, and the `F` bit is cleared by calling the `ARM_TIM_clear_INT_ASM` subroutine before branching to the polling loop.

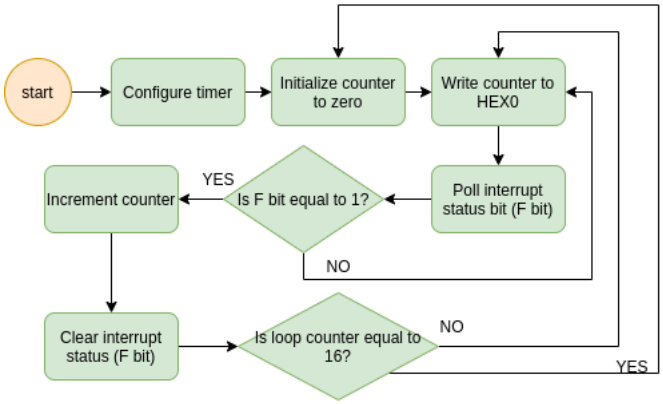


Figure 1: Up counter application flowchart

Polling-based stopwatch

The polling-based stopwatch counts in increments of 10ms and uses `PB0`, `PB1`, and `PB2` to start, stop, and reset the timer respectively. `HEX0-1` display milliseconds, `HEX2-3` display seconds, and `HEX4-5` display minutes. Figure 2 shows the flowchart of the polling-based stopwatch.

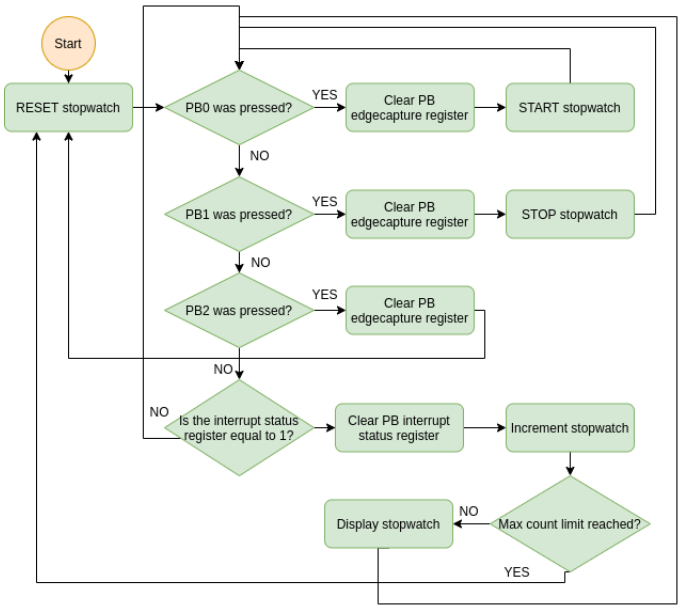


Figure 2: Stopwatch application flowchart

Similar to the simple up counter program, we use counters to keep track of time with the difference that we associate one counter to each hexadecimal display. For example, counter registers `R3, R4, R5, R6, R7, R8` keep track of centiseconds, deciseconds, seconds, decaseconds, minutes, and decaminutes respectively. Similar to the up counter, the application runs in a polling loop where we call the start, stop, and reset subroutines if their corresponding pushbuttons have been pushed and released by polling the pushbutton edge capture register using the `PB_edgecp_is_pressed_ASM` subroutine. The reset function simply sets all counter registers to zero, while the start and stop procedures configure the timer, by setting the `E` bit of the control register to 1 to start the timer and to 0 to stop it. As the timer runs at a clock frequency of 200MHz, we set the initial count to 2'000'000 to keep track of time with a precision of 10 milliseconds. We also poll the `F` bit of the timer interrupt status register and increment the counter value by 1 and write to the HEX displays if

the F bit is equal to 1 (meaning that 10ms have passed). We also clear both polled registers if they contains a value other than zeros.

Incrementing the counters is different from the up-counter described in the previous section. We increment the value of the centisecond register by 1. However, if the value passes 9 after incrementing, we set the centiseconds register to 0 and increment the deciseconds register by 1 and continue this operation for all counter register. This process is slightly different for the decaseconds and the decaminute registers as a minute are 60 seconds, and an hour is 60 minutes. If the value passes 60 minutes (the maximum we can display), we reset the timer. We then write each counter register to their corresponding HEX display using `HEX_write_ASM`.

We test the counter and stopwatch applications by running them and assuring correct timing using a timer on our phones. Additionally, we test the stopwatch by starting, stopping, and resetting them using their corresponding pushbuttons in random order and assure expected behaviour. The main challenge faced in this part was understanding polling and making the code concise and readable.

3 Interrupts

Interrupt-based stopwatch

To implement the interrupt-based stopwatch, we modify our polling-based stopwatch application and add code to the template code to handle interrupts from the A9 private timer and the pushbuttons.

We first initialize the vector containing branch instructions to the address of the relevant exception handler. In start, we set the stack pointers for the IRQ and SVC processor mode. Additionally, we enable interrupts for pushbuttons calling `enable_PB_INT_ASM` and the timer by calling `ARM_TIM_config_ASM` and setting the `I` bit in the control register to 1. We modify `SERVICE_IRQ` to call the appropriate ISR if an interrupt has occurred by checking the interrupt IDs (73 for pushbuttons, 29 for the timer). Then we configure the GIC by set the target to cpu0 in the ICDIPTRn register and enable timer and pushbutton interrupts in the ICDISERn register by passing their IDs. For the pushbutton ISR (`KEY_ISR`), we read the pushbutton edge capture register by calling the `read_PB_edgecp_ASM` and storing the value in the `PB_int_flag memory` to use in the stopwatch application. Finally, we clear the pushbutton edge capture register by calling the `PB_clear_edgecp_ASM subroutine`. The ISR for the A9 private timer (`ARM_TIM_ISR`) is similar to the `KEY_ISR` with the difference that the `F` bit of the interrupt status register is read, cleared, and stored in the `TIM_int_flag` memory location by calling `ARM_TIM_read_INT_ASM` and `ARM_TIM_clear_INT_ASM`.

We write the stopwatch application under the `IDLE` label where it will run until an interrupt occurs. The only difference between the interrupt-based stopwatch and the polling-based stopwatch is that the interrupt-based stopwatch polls the value in `PB_int_flag` and `TIM_int_flag` memory addresses instead of the pushbutton edge capture (using `PB_edgecp_is_pressed_ASM`) and the timer interrupt status register (using `ARM_TIM_read_INT_ASM`). `PB_int_flag` and `TIM_int_flag` memory addresses are cleared once they read. Figure 2 with the mentioned modifications applies to the interrupt-based stopwatch. As we are still polling values in a loop, using interrupts in our case, does not improve the performance of our application. The main challenge in this part was understanding interrupts and configuring the GIC that was overcome by reading the appropriate documentation. We tested the program in the same way as the polling-based stopwatch.

4 Conclusion

In conclusion, in this lab, we implemented drivers for I/O devices such as slider switches, LEDs, pushbuttons, HEXadecimal displays, and the ARM A9 private driver. We used the written drivers to write multiple programs such as a programmable display, an up counter, and a stopwatch. We implemented the stopwatch application, once using polling and a second time using interrupts.